Name of Student: VIKAS FOOLCHAND KUSHWAHA			
Roll Number: 29/B		**Lab Practical Number:** 01	
Title of LAB Practical: Implement the Basic R commands.			
DOP: 06/11/2026		**DOS:** 12/11/2026	
CO Mapped: CO1	**PO Mapped:** PO1, PO5, PO7, PSO1	**Faculty Signature:**	**Marks:**

AIM:

To install R and implement basic commands in R.

Theory:

Introduction to R commands

R commands are instructions you give R to work with data, perform calculations, or display results. You enter them in the R console or save them in a script and run them.

- Assign values: Store a value or result under a name so you can use it later.
- Create objects: R can store data as vectors, matrices, lists, factors, data frames, and tables. Each structure is suited to different kinds of information.
- Inspect objects: Commands can show an object's contents, structure, dimensions, or data type.
- Calculate and transform: Use commands to perform arithmetic, summarize data, filter rows, or create new variables.
- Manage your workspace: You can check the current working directory, change where R looks for files, list available datasets, or remove objects you no longer need.
- Work with packages: Install packages to add capabilities, then load them in a session to use their functions.
- Import and export data: R can read data from sources such as CSV and Excel files, and save results to files.
- Combine data: Commands can join data side by side by columns or stack compatible data by rows.

R is case-sensitive, so names with different capitalization are treated as different names. Most commands operate on objects and return a result; you can assign that result to a name or display it in the console.

Code:

1. Install Packages.

CODE

`# 1. Install packages`
`# Install a package once (or when you need to update it).`
`install.packages("readxl")`
`install.packages("writexl")`

OUTPUT

```
> # 1. Install packages
> # Install a package once (or when you need to update it).
> install.packages("readxl")
```

```
Installing package into '/home/vikas/.local/share/R/x86_64-pc-linux-gnu-library'
(as 'lib' is unspecified)
```

```
trying URL 'https://cran.rstudio.com/src/contrib/readxl_1.5.0.1.tar.gz'
Content type 'application/x-gzip' length 2089119 bytes (2.0 MB)
==================================================
downloaded 2.0 MB

* installing *source* package 'readxl' ...
** this is package 'readxl' version '1.5.0.1'
** package 'readxl' successfully unpacked and MD5 sums checked
** using staged installation
** libs
using C compiler: 'gcc (GCC) 16.2.1 20260810'
using C++ compiler: 'g++ (GCC) 16.2.1 20260810'
g++ -std=gnu++20 -I"/usr/include/R/" -DNDEBUG -Iunix -I. -I'/home/vikas/.local/share/R/x86_64-pc
linux-gnu-library/cpp11/include' -I'/home/vikas/.local/share/R/x86_64-pc-linux-gnu-library/progr
ss/include' -I/usr/local/include   -fvisibility=hidden -fpic  -march=x86-64 -mtune=generic -O2 -
ipe -fno-plt -fexceptions         -Wp,-D_FORTIFY_SOURCE=3 -Wformat -Werror=format-security
-fstack-clash-protection -fcf-protection         -fno-omit-frame-pointer -mno-omit-leaf-frame-po
nter -Wp,-D_GLIBCXX_ASSERTIONS -g -ffile-prefix-map=/build/r/src=/usr/src/debug/r -flto=auto -ff
t-lto-objects   -c cpp11.cpp -o cpp11.o
```

2. Load packages

CODE

```
# 2. Load packages
library(readxl)
library(writexl)

# Alternatively, call a package function without attaching the
package:
readxl::read_excel("DALP1/grades.xlsx")
```

OUTPUT

```
> # 2. Load packages
> library(readxl)
> library(writexl)
>
```

```
> # Alternatively, call a package function without attaching the package:
> readxl::read_excel("DALP1/grades.xlsx")
# A tibble: 4 × 2
  Name   Grade
  <chr>  <chr>
1 Ramesh A
2 Rohini B
3 Rakesh A
4 Rohan  C
>
```

3. Data types and checking variable types

CODE

```
# 3. Data types and checking variable types
number <- 42
decimal <- 3.14
text <- "hello"
is_ready <- TRUE

class(number)        # "numeric"
typeof(number)       # underlying storage type
str(text)            # compact structure
is.numeric(number)   # TRUE
is.character(text)   # TRUE
is.logical(is_ready) # TRUE

# Convert between common types
as.character(number)
as.numeric("12.5")
as.logical(1)       # TRUE
```

OUTPUT

```
> # 3. Data types and checking variable types
> number <- 42
> decimal <- 3.14
> text <- "hello"
> is_ready <- TRUE
> class(number)        # "numeric"
[1] "numeric"
> typeof(number)       # underlying storage type
[1] "double"
> str(text)            # compact structure
 chr "hello"
> is.numeric(number)   # TRUE
[1] TRUE
> is.character(text)   # TRUE
[1] TRUE
> is.logical(is_ready) # TRUE
[1] TRUE
> # Convert between common types
> as.character(number)
[1] "42"
> as.numeric("12.5")
[1] 12.5
> as.logical(1)       # TRUE
[1] TRUE
>
```

4. Create and print common objects

CODE

`# 4. Create and print common objects`
 	
`# Vector: one-dimensional; elements generally share a type`
`my_vector <- c(10, 20, 30)`
`print(my_vector)`

```r
# Matrix: two-dimensional; elements share a type
my_matrix <- matrix(1:6, nrow = 2, ncol = 3)
print(my_matrix)

# List: can contain different types and object sizes
my_list <- list(name = "Vikas", age = 22, scores = c(85, 92))
print(my_list)

# Factor: represents categorical data
my_factor <- factor(c("low", "high", "low"),
                    levels = c("low", "high"))
print(my_factor)
levels(my_factor)

# Data frame: columns can have different types
my_data_frame <- data.frame(
  name = c("Vikas", "Ganesh"),
  age = c(22, 25),
  passed = c(TRUE, FALSE)
)
print(my_data_frame)

# Table: frequency counts
my_table <- table(c("red", "blue", "red", "green", "red"))
print(my_table)
```

OUTPUT

```
> # 4. Create and print common objects
>
> # Vector: one-dimensional; elements generally share a type
> my_vector <- c(10, 20, 30)
> print(my_vector)
[1] 10 20 30
>
> # Matrix: two-dimensional; elements share a type
> my_matrix <- matrix(1:6, nrow = 2, ncol = 3)
> print(my_matrix)
     [,1] [,2] [,3]
[1,]    1    3    5
[2,]    2    4    6
>
> # List: can contain different types and object sizes
> my_list <- list(name = "Vikas", age = 22, scores = c(85, 92))
> print(my_list)
$name
[1] "Vikas"

$age
[1] 22

$scores
[1] 85 92

>
```

```
> # Factor: represents categorical data
> my_factor <- factor(c("low", "high", "low"),
+                     levels = c("low", "high"))
> print(my_factor)
[1] low  high low
Levels: low high
> levels(my_factor)
[1] "low"  "high"
>
> # Data frame: columns can have different types
> my_data_frame <- data.frame(
+   name = c("Vikas", "Ganesh"),
+   age = c(22, 25),
+   passed = c(TRUE, FALSE)
+ )
> print(my_data_frame)
    name age passed
1  Vikas  22   TRUE
2 Ganesh  25  FALSE
>
> # Table: frequency counts
> my_table <- table(c("red", "blue", "red", "green", "red"))
> print(my_table)

 blue green   red
    1     1     3
>
```

5. Combine data by columns or rows

CODE

`# 5. Combine data by columns or rows`
`left <- data.frame(id = 1:3, name = c("A", "B", "C"))`
`right <- data.frame(score = c(80, 90, 85))`
 	
`combined_columns <- cbind(left, right)`
`print(combined_columns)`

```
more_rows <- data.frame(id = 4:5, name = c("D", "E"))
combined_rows <- rbind(left, more_rows)
print(combined_rows)
```

OUTPUT

```
> # 5. Combine data by columns or rows
> left <- data.frame(id = 1:3, name = c("A", "B", "C"))
> right <- data.frame(score = c(80, 90, 85))
>
> combined_columns <- cbind(left, right)
> print(combined_columns)
  id name score
1  1    A    80
2  2    B    90
3  3    C    85
>
> more_rows <- data.frame(id = 4:5, name = c("D", "E"))
> combined_rows <- rbind(left, more_rows)
> print(combined_rows)
  id name
1  1    A
2  2    B
3  3    C
4  4    D
5  5    E
>
```

6. Working directory

CODE

```
# 6. Working directory
getwd()                    # Show current working directory
```

```
setwd("~/Dev/labs/DALP")  # For Unix based machines
```

OUTPUT

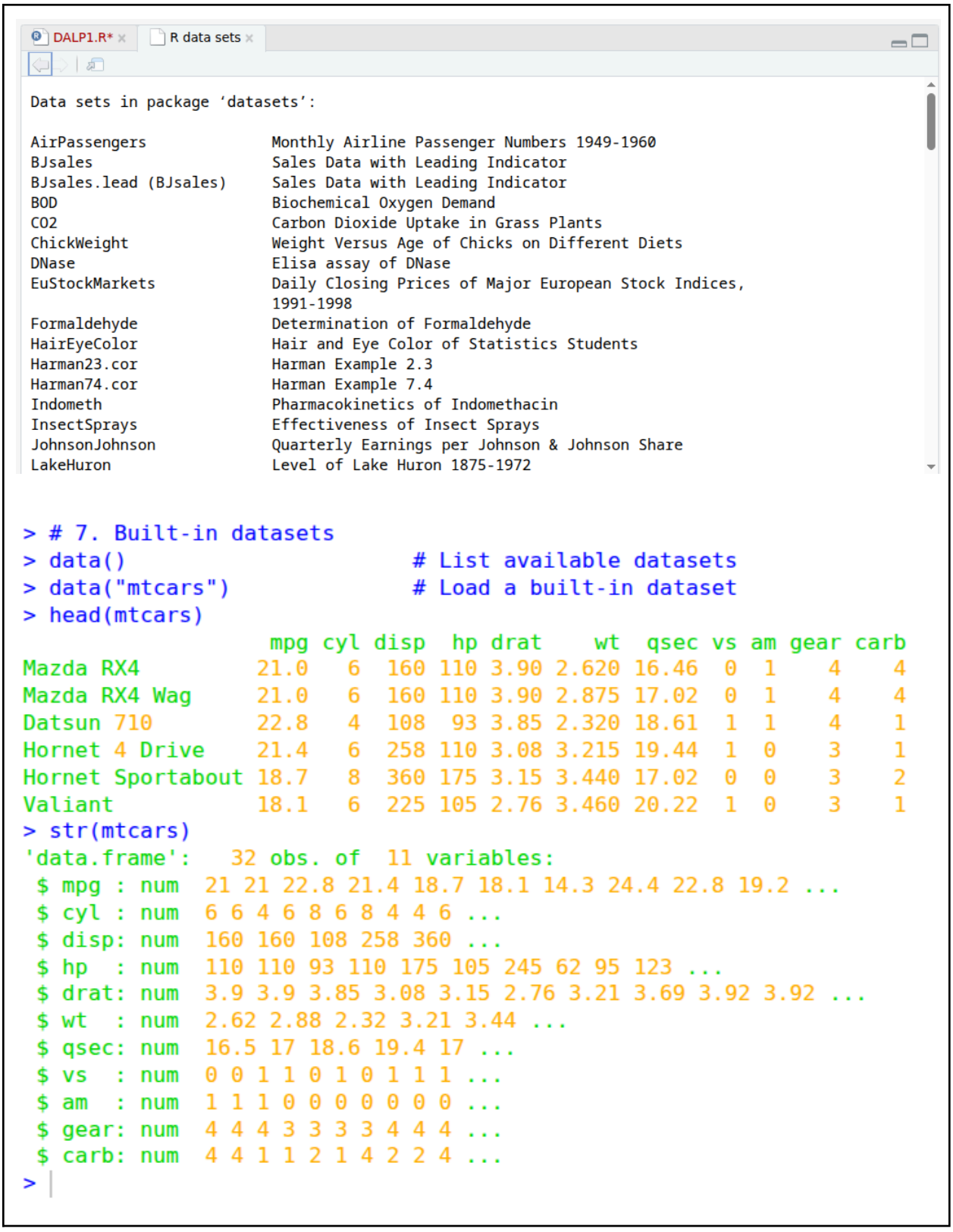
```
> getwd()                       # Show current working directory
[1] "/home/vikas/Dev/labs/DALP"
> setwd("~/Dev/labs/DALP")  # For Unix based machines
>
```

7. Built-in datasets

CODE

```
# 7. Built-in datasets
data()                        # List available datasets
data("mtcars")                # Load a built-in dataset
head(mtcars)
str(mtcars)
```

OUTPUT


DALP1.R*
R data sets
Data sets in package 'datasets':
AirPassengers Monthly Airline Passenger Numbers 1949-1960
BJsales Sales Data with Leading Indicator
BJsales.lead (BJsales) Sales Data with Leading Indicator
BOD Biochemical Oxygen Demand
CO2 Carbon Dioxide Uptake in Grass Plants
ChickWeight Weight Versus Age of Chicks on Different Diets
DNase Elisa assay of DNase
EuStockMarkets Daily Closing Prices of Major European Stock Indices, 1991-1998
Formaldehyde Determination of Formaldehyde
HairEyeColor Hair and Eye Color of Statistics Students
Harman23.cor Harman Example 2.3
Harman74.cor Harman Example 7.4
Indometh Pharmacokinetics of Indomethacin
InsectSprays Effectiveness of Insect Sprays
JohnsonJohnson Quarterly Earnings per Johnson & Johnson Share
LakeHuron Level of Lake Huron 1875-1972
> # 7. Built-in datasets
> data() # List available datasets
> data("mtcars") # Load a built-in dataset
> head(mtcars)
mpg cyl disp hp drat wt qsec vs am gear carb
Mazda RX4 21.0 6 160 110 3.90 2.620 16.46 0 1 4 4
Mazda RX4 Wag 21.0 6 160 110 3.90 2.875 17.02 0 1 4 4
Datsun 710 22.8 4 108 93 3.85 2.320 18.61 1 1 4 1
Hornet 4 Drive 21.4 6 258 110 3.08 3.215 19.44 1 0 3 1
Hornet Sportabout 18.7 8 360 175 3.15 3.440 17.02 0 0 3 2
Valiant 18.1 6 225 105 2.76 3.460 20.22 1 0 3 1
> str(mtcars)
'data.frame': 32 obs. of 11 variables:
$ mpg : num 21 21 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 ...
$ cyl : num 6 6 4 6 8 6 8 4 4 6 ...
$ disp: num 160 160 108 258 360 ...
$ hp : num 110 110 93 110 175 105 245 62 95 123 ...
$ drat: num 3.9 3.9 3.85 3.08 3.15 2.76 3.21 3.69 3.92 3.92 ...
$ wt : num 2.62 2.88 2.32 3.21 3.44 ...
$ qsec: num 16.5 17 18.6 19.4 17 ...
$ vs : num 0 0 1 1 0 1 0 1 1 1 ...
$ am : num 1 1 1 0 0 0 0 0 0 0 ...
$ gear: num 4 4 4 3 3 3 3 4 4 4 ...
$ carb: num 4 4 1 1 2 1 4 2 2 4 ...
>

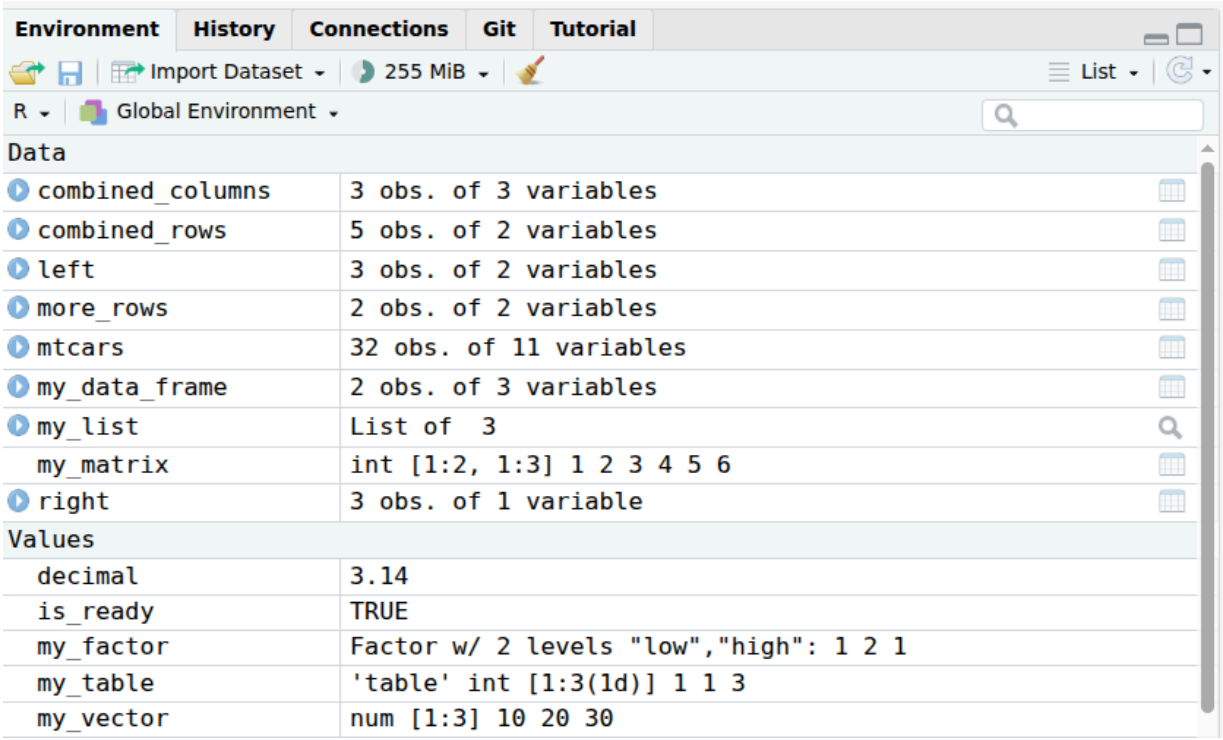
8. Remove objects

CODE

```
# 8. Remove objects
temporary_object <- 123
rm(temporary_object)        # Remove one object
rm(list = ls())            # Remove all objects in the workspace
```

OUTPUT

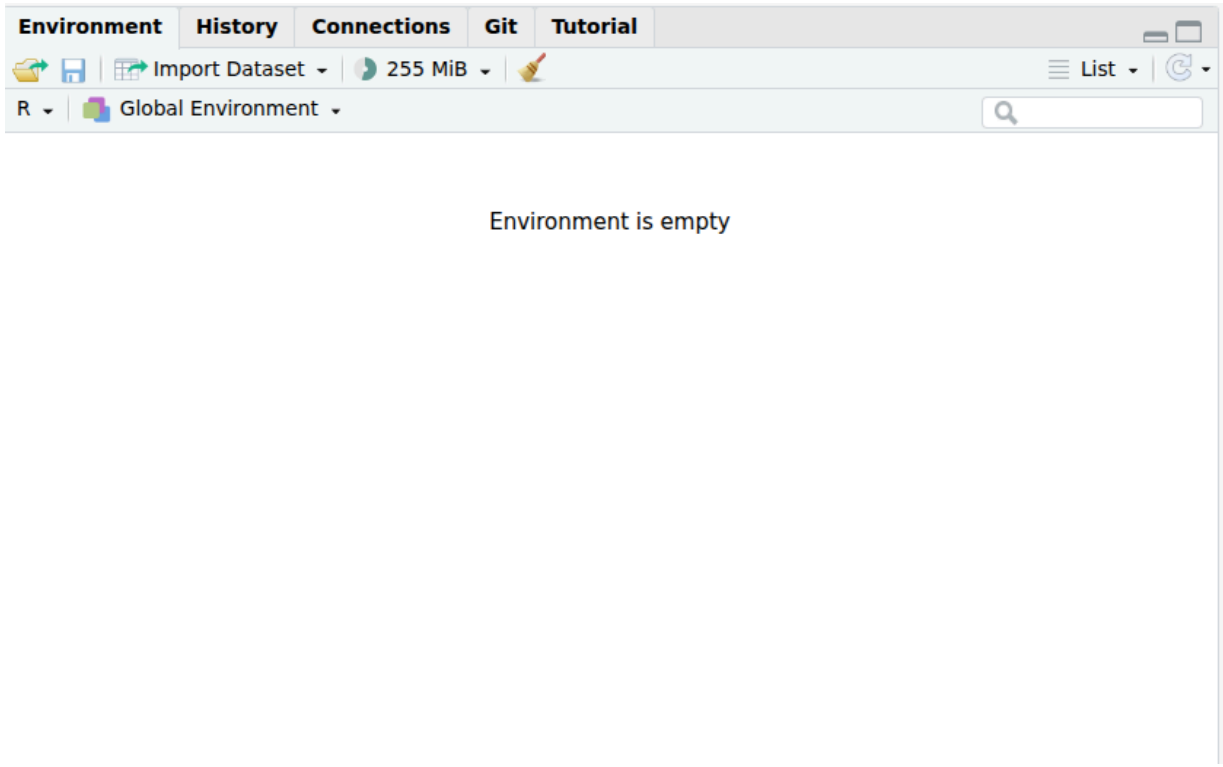
Check Environment Tab:



Execute code:

```
> # 8. Remove objects
> temporary_object <- 123
> rm(temporary_object)        # Remove one object
> rm(list = ls())            # Remove all objects in the workspace
>
```

Recheck Environment Tab:



9. Attach and detach data

CODE

```
# 9. Attach and detach data
# Attaching lets you refer to columns by name, but can cause name conflicts.
attach(mtcars)
mean(mpg)
detach(mtcars)

# Safer alternative: refer to columns through the data frame.
mean(mtcars$mpg)
```

OUTPUT

```
> # 9. Attach and detach data
> # Attaching lets you refer to columns by name, but can cause name conflicts.
> attach(mtcars)
> mean(mpg)
[1] 20.09062
> detach(mtcars)
>
> # Safer alternative: refer to columns through the data frame.
> mean(mtcars$mpg)
[1] 20.09062
>
```

10. Read data from the console

CODE

```
# 10. Read data from the console
# Type values at the prompt, then press Enter.
console_values <- scan()

# Read one line of text:
console_text <- readline("Enter a value: ")
```

OUTPUT

```
> # 10. Read data from the console
> # Type values at the prompt, then press Enter.
> console_values <- scan()
1: 25
2: 22
3: 33
4: 24
5:
Read 4 items
> # Read one line of text:
> console_text <- readline("Enter a value: ")
Enter a value: 22
>
```

11. Read and write CSV files

CODE

```
# 11. Read and write CSV files
# Read a CSV file
csv_data <- read.csv("DALP1/students.csv", header = TRUE)

# Write a data frame to a CSV file
write.csv(csv_data, "output.csv", row.names = FALSE)
```

OUTPUT

DALP1.R* | students.csv | output.csv

```
1  Roll No,Name,Age
2  1,Vikas,23
3  2,Ramesh,23
4  3,Aryan,20
5  4,Aayush,21
6  5,Uday,22
7
```

```
> # 11. Read and write CSV files
> # Read a CSV file
> csv_data <- read.csv("DALP1/students.csv", header = TRUE)
>
> # Write a data frame to a CSV file
> write.csv(csv_data, "output.csv", row.names = FALSE)
>
```

DALP1.R* | students.csv | output.csv

```
1  "Roll.No","Name","Age"
2  1,"Vikas",23
3  2,"Ramesh",23
4  3,"Aryan",20
5  4,"Aayush",21
6  5,"Uday",22
7
```

12. Read and write Excel files

CODE

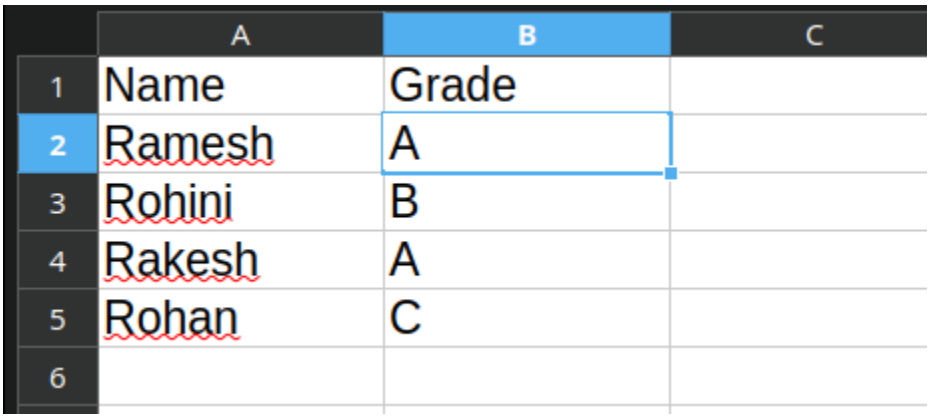
```
# 12. Read and write Excel files
# Read the first worksheet in an Excel file
excel_data <- read_excel("DALP1/grades.xlsx")

# Read a named worksheet
excel_data <- read_excel("DALP1/grades.xlsx", sheet = "Sheet1")

# Write a data frame to an Excel file
write_xlsx(excel_data, "DALP1/output.xlsx")
```

OUTPUT

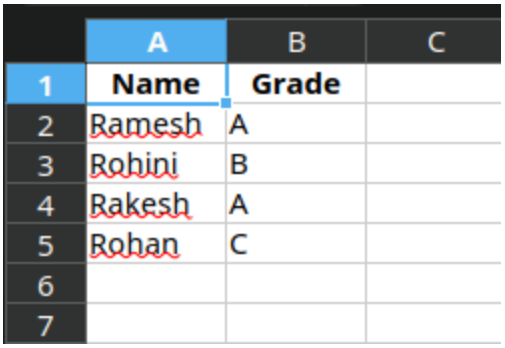
grades.xlsx



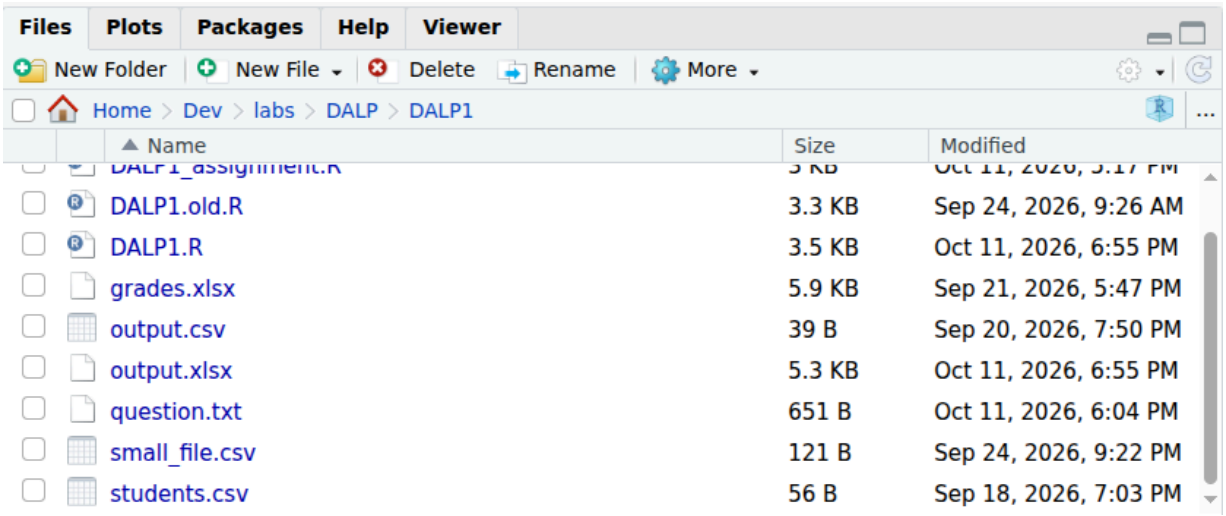
	A	B	C
1	Name	Grade	
2	Ramesh	A	
3	Rohini	B	
4	Rakesh	A	
5	Rohan	C	
6			

```
> # 12. Read and write Excel files
> # Read the first worksheet in an Excel file
> excel_data <- read_excel("DALP1/grades.xlsx")
>
> # Read a named worksheet
> excel_data <- read_excel("DALP1/grades.xlsx", sheet = "Sheet1")
>
> # Write a data frame to an Excel file
> write_xlsx(excel_data, "DALP1/output.xlsx")
>
```

output.xlsx



	A	B	C
1	Name	Grade	
2	Ramesh	A	
3	Rohini	B	
4	Rakesh	A	
5	Rohan	C	
6			
7			



13. Other common data sources

CODE

```
# 13. Other common data sources
# Tab-delimited text file:
tab_data <- read.table("DALP1/students.tsv", header = TRUE)
tab_data

# R binary formats:

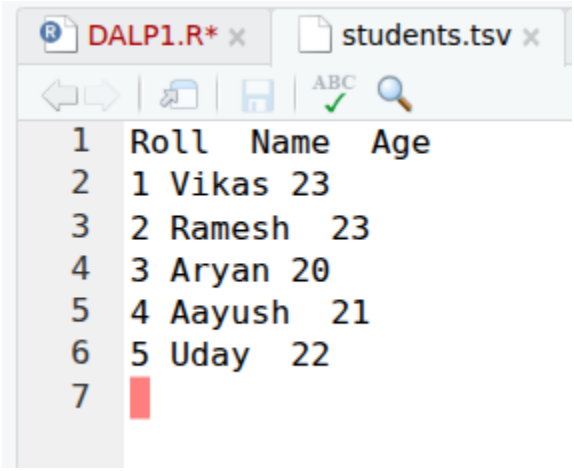
# Data frame
my_data_frame <- data.frame(
```

```
  name = c("Vikas", "Ganesh"),
  age = c(22, 25),
  passed = c(TRUE, FALSE)
)
print(my_data_frame)

save(my_data_frame, file = ".RData")
load(".RData")

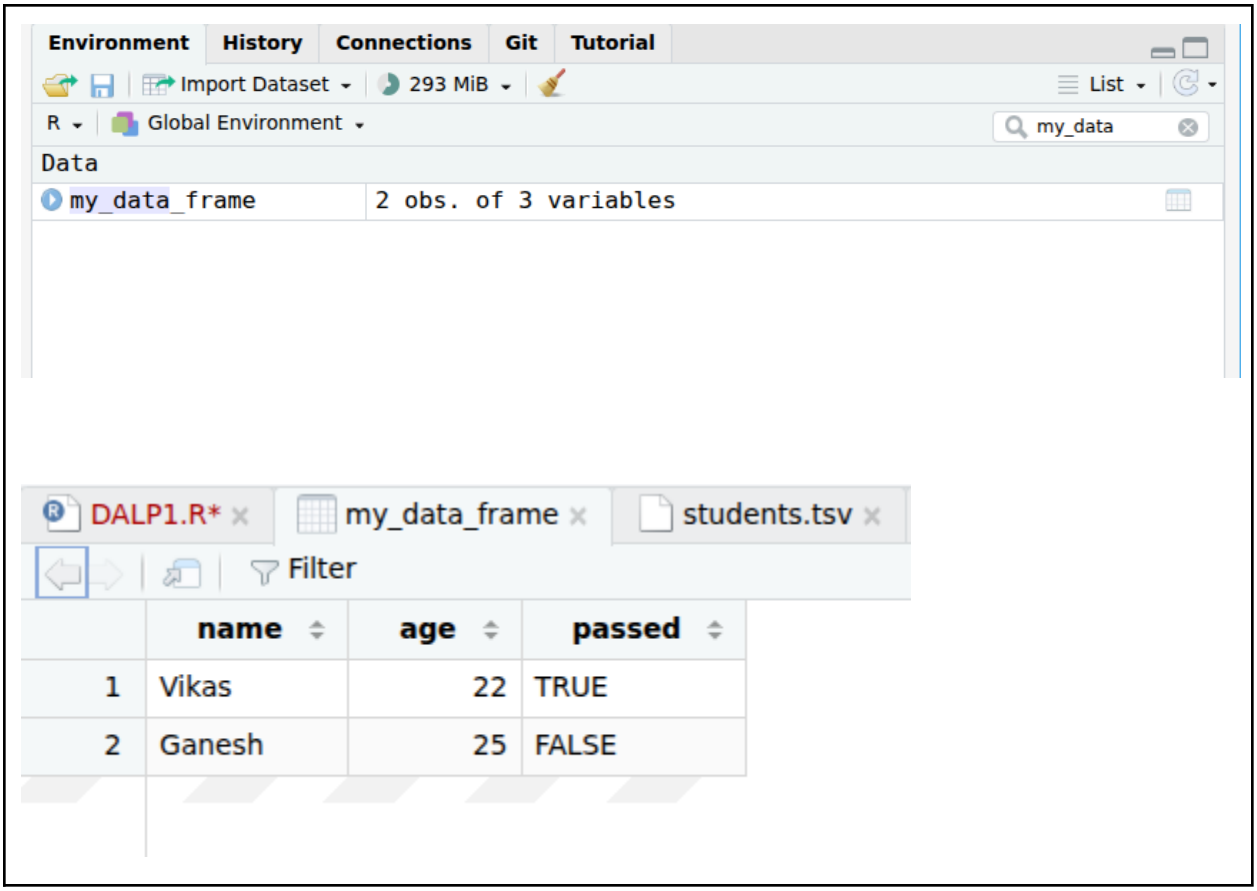
saveRDS(my_data_frame, "data.rds")
restored_data <- readRDS("data.rds")
restored_data
```

OUTPUT



```
> # 13. Other common data sources
> # Tab-delimited text file:
> tab_data <- read.table("DALP1/students.tsv", header = TRUE)
> tab_data
  Roll   Name Age
1    1  Vikas  23
2    2 Ramesh  23
3    3  Aryan  20
4    4 Aayush  21
5    5   Uday  22
>
```

```
> # Data frame
> my_data_frame <- data.frame(
+   name = c("Vikas", "Ganesh"),
+   age = c(22, 25),
+   passed = c(TRUE, FALSE)
+ )
> print(my_data_frame)
    name age passed
1  Vikas  22   TRUE
2 Ganesh  25  FALSE
>
> save(my_data_frame, file = ".RData")
> load(".RData")
>
> saveRDS(my_data_frame, "data.rds")
> restored_data <- readRDS("data.rds")
> restored_data
    name age passed
1  Vikas  22   TRUE
2 Ganesh  25  FALSE
```



Conclusion:

In this practical, we learned about Basic R commands. The practical initially started with installing and importing libraries, using built-in data types, storing simple data, etc. and eventually grew into complex operations like running file system operations, attaching and detaching data, reading from console and reading from various kinds of external files and data sources.